Member's Group:

- 223026679 Janvier NSHIMIYIMANA
- 223036675 Blandon ASHIMIRWE
- 223026689 Gilbert HIRWA NTWALI

COMPUTER ENGINEERING LEVEL_3

GAKO COMPUS

MOBILE APPLICATION

On 18th April, 26

MOBILE APP LAB6 PUSH NOTIFICATION

Push notifications have become a fundamental feature in modern mobile applications, enabling real-time communication between systems and users. They allow applications to deliver important information instantly, even when the application is not actively in use. This improves user engagement, responsiveness, and overall user experience.

In this lab, the focus is on implementing push notifications using **Firebase Cloud Messaging (FCM)** within a Flutter mobile application. FCM is a cloud-based messaging service provided by Google that allows developers to send notifications and data messages to devices in a reliable and efficient manner.

The lab demonstrates how to integrate Firebase into a Flutter project, request user permission for notifications, retrieve a unique device token, and handle incoming messages in different application states (foreground and background). Additionally, it explores how notifications can be displayed both in the system notification tray and within the application interface.

By completing this lab, a practical understanding is gained on how mobile applications communicate with cloud services to deliver real-time updates, which is an essential concept in modern app development.

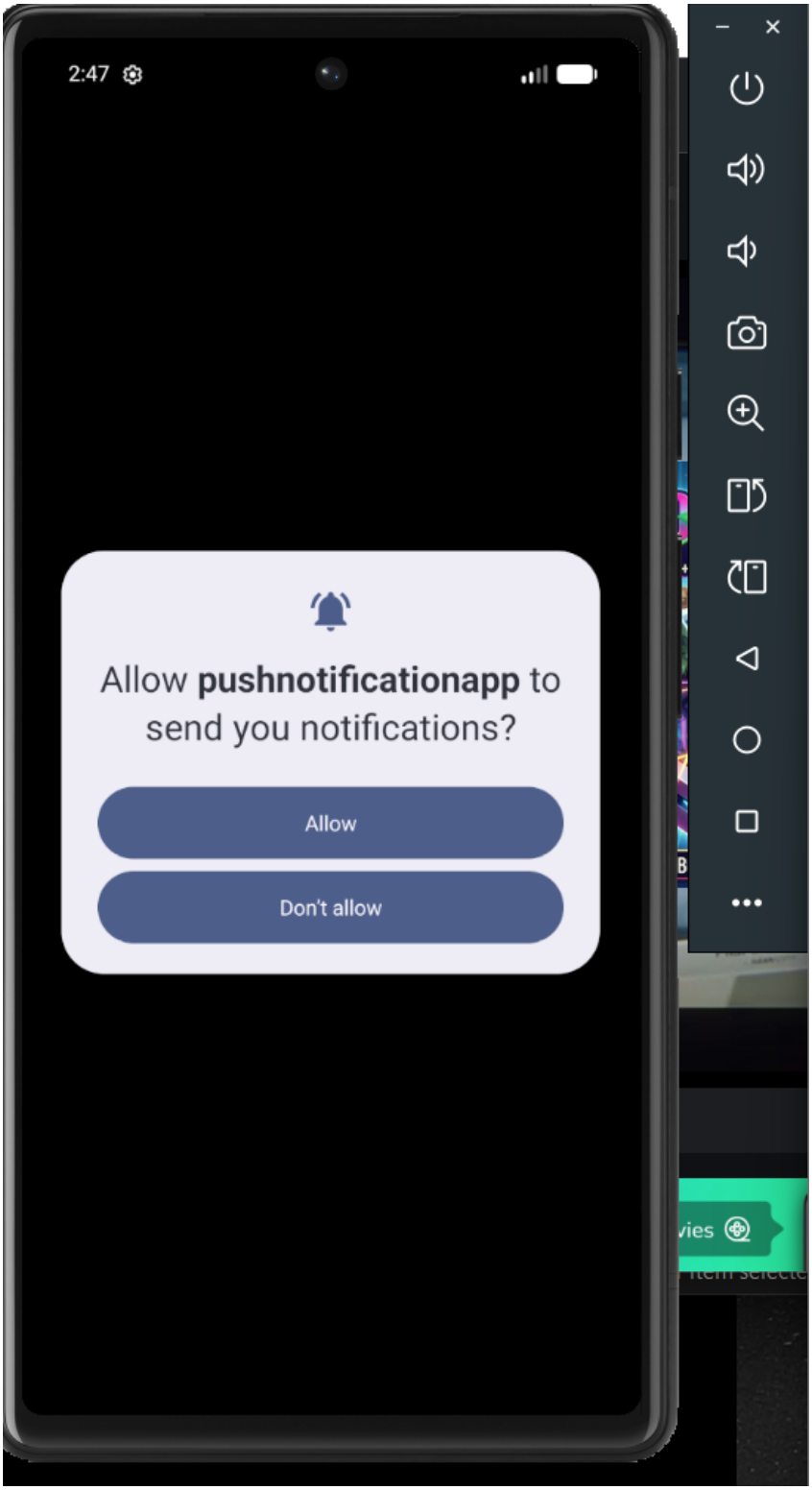

Figure1: Allow Notification on device

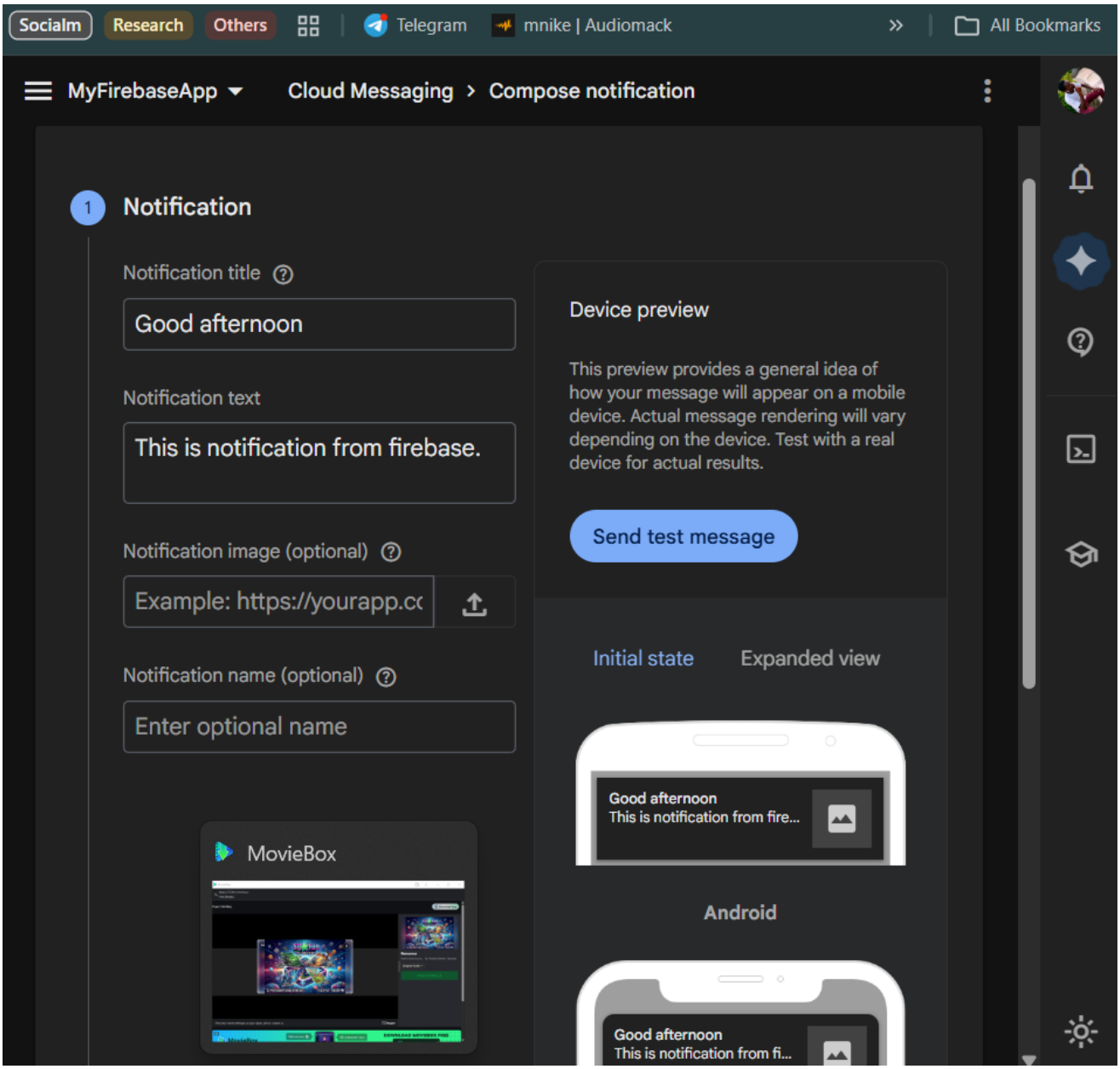

Figure2: FCM notification details setting

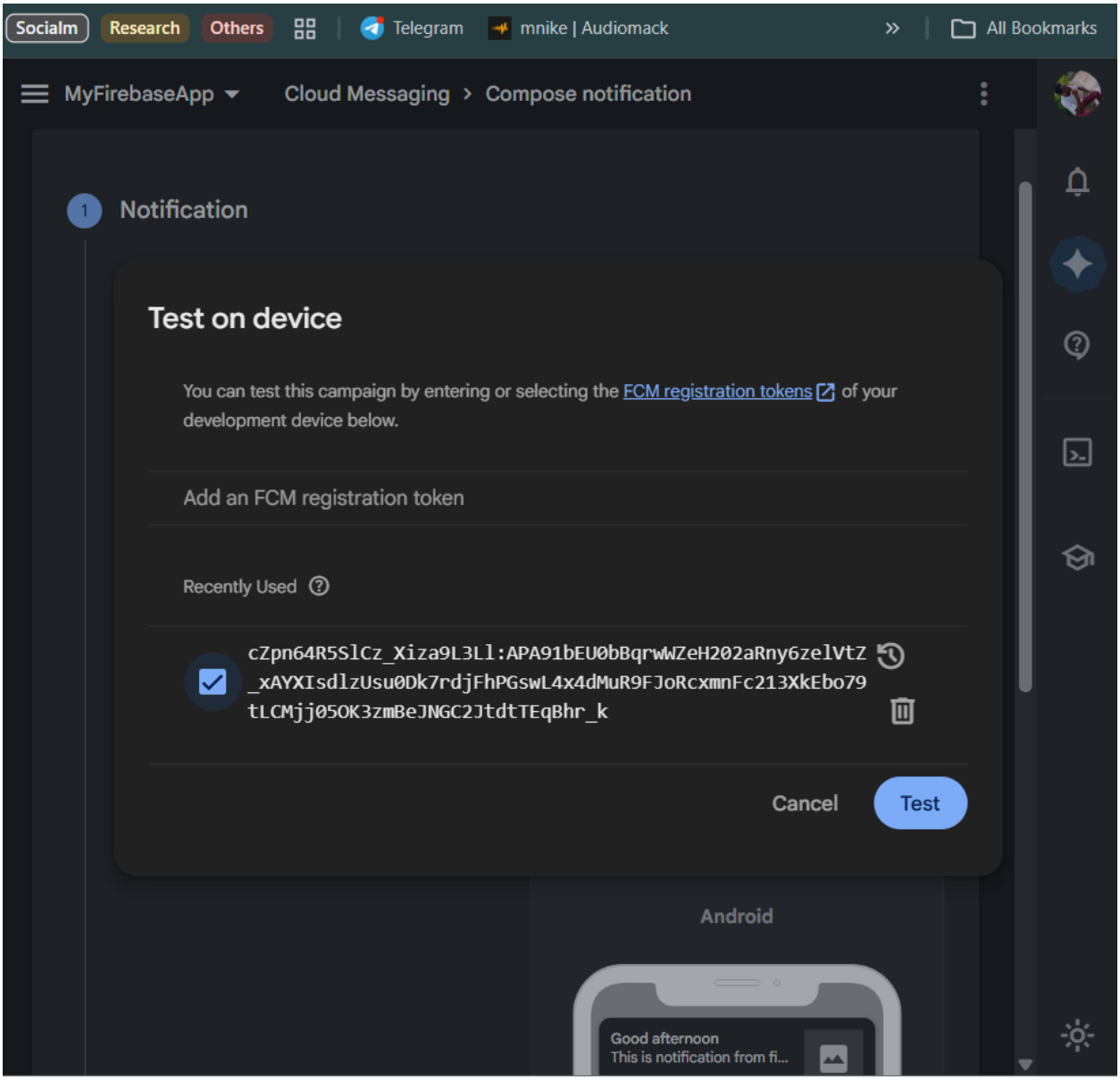

Figure3: Add an FCM registration device-token

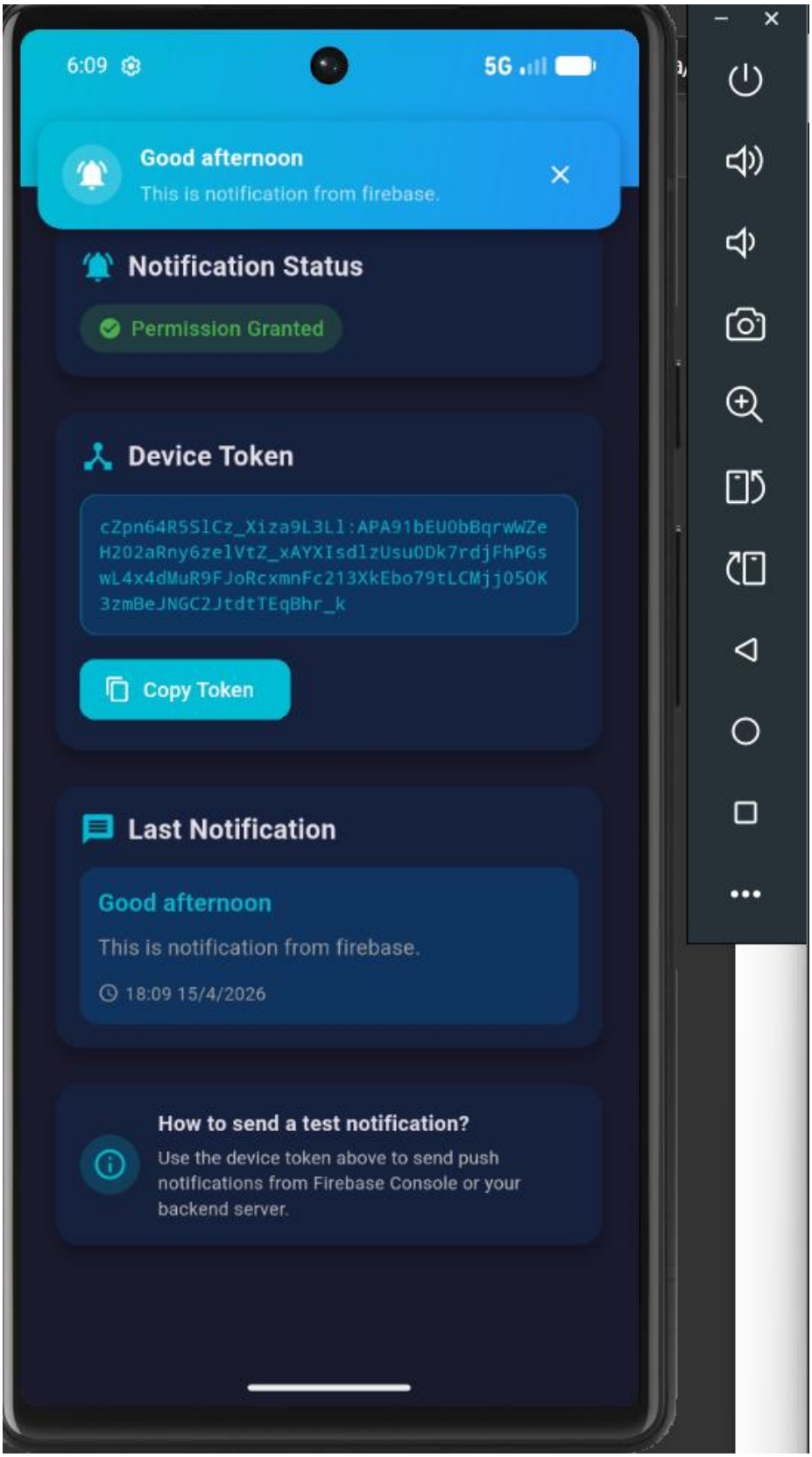

Figure4: Notification received in Foreground

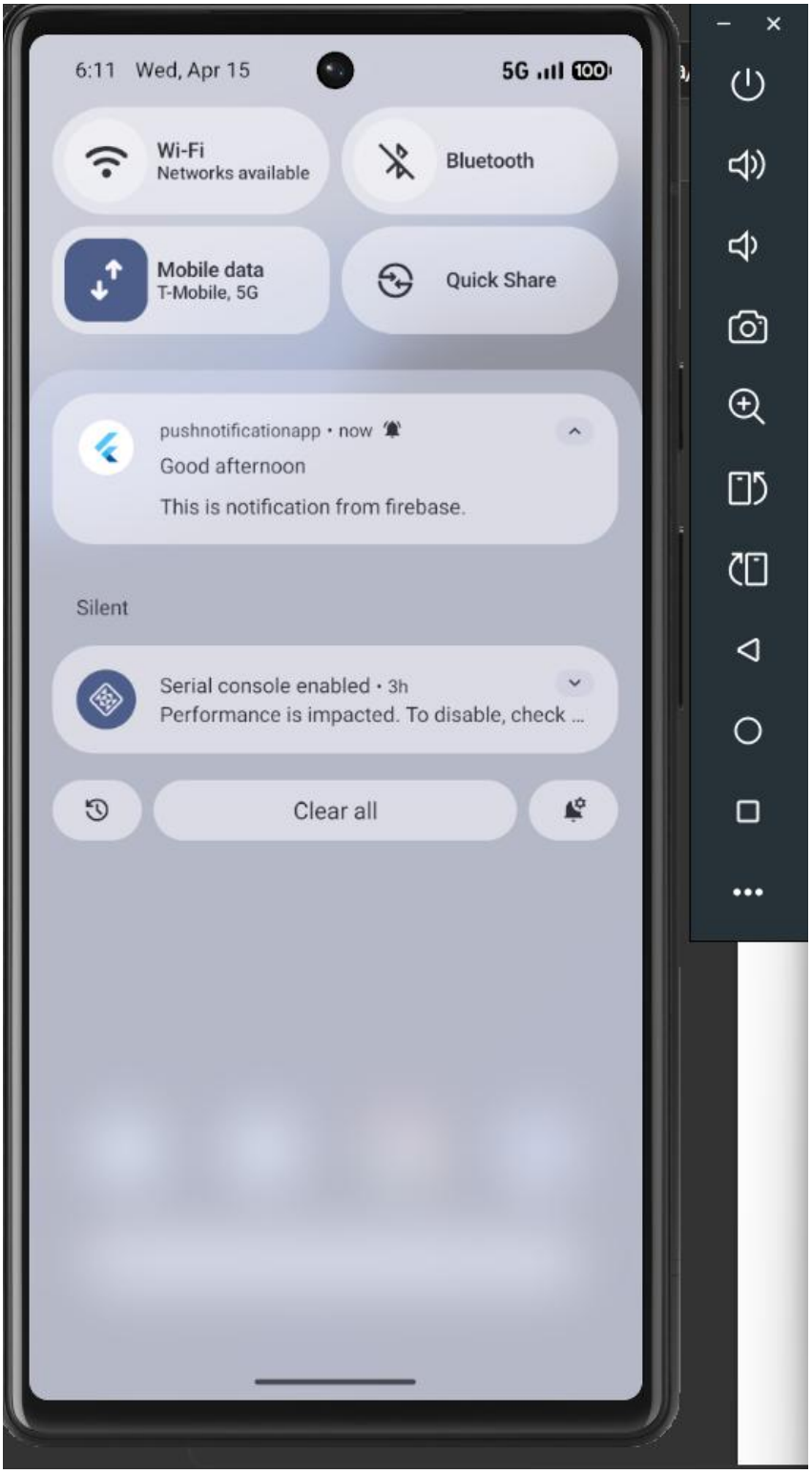

Figure5: Device popup notification in Background

Members:
- 223026679
- 223026675
- 223026689

Gako_Campus
Computer Department
On 18th Apr, 26

Lab 6 Push Notification.

⊛ Observation on this Lab 6:

During the implementation of push notifications using FCM, several important behaviors and system characteristics were observed:

- → The application successfully requested and obtained user permission to display notifications, which is a critical step before any message can be received.
- → Each device generate a unique registration token. This token was essential for targeting a specific device when sending notifications from the firebase console.
- → Notifications were received differently depending on the application state:
  - Foreground: Messages were captured programmatically and displayed inside the app UI, along with a popup dialog.
  - Background: Notifications appeared directly in the system notification tray without interrupting the user interface.
- → Messages sent from the firebase console were delivered almost instantly, demonstrating the real-time capability and reliability of FCM.
- → The notification content (title and body) was correctly extracted from the received message object and displayed within the application.
- → When the user tapped a notification from the system tray, the application was opened and the corresponding event listener was triggered.

→ Testing showed that push notifications work more reliably on a real physical device compared to an emulator, especially for background and terminated states.

→ Proper configuration of Firebase and permissions is crucial, missing steps can prevent notifications from being received.

● Key steps Followed.

To successfully implement push notification using FCM in the flutter application, the following structured steps were followed:

1. Created the flutter Application:
   initialized a new flutter project and ensure the environment was properly configured.

2. Integrated Firebase into the project:
   Connected the app to firebase by adding the project in the firebase console and configuring it with the flutter app (including downloading and placing the required configuration files).

3. Added Required Dependencies
   Included firebase_core and firbase_messaging in the pubspec.yaml file to enable firebase services and messaging functionality.

4. Initialized Firbase in main.dart
   Ensured firebase was initialized before running the application using Firebase.initializeApp();

5. Requested Notification permissions
   implemented permission requests to allow the app to display notifications to the user.

6. Retrieved the Device token:
   Used FCM to generate a unique device token and displayed/printed it for used in sending targeted notifications.

7. Configured Message Handlers
   - ~~set up a background listener using FirebaseMessaging.onMessag~~
   - set up a background message handle.
   - implemented a foreground listener using FirebaseMessaging.onMessag
   - Handled notification click events using onMessageOpenedApp.

8. Displayed Notifications in the App UI:

Updated the interface dynamically to show incoming messages and implemented a popup dialog to notify users in real time.

9. Configured Android settings.

Updated AndroidManifest.Xml to include required permissions and notification channel configuration.

10. Sent Test Notifications from Firbase Console.

Navigated to firebase Console → Cloud Messaging → Create a test message → sent it using the device token.

11. Tested in Different App states.

Verified functionality when the app was:

- opened (foreground)
- Running in the background
- Closed and opened via notification.

To conclude, this lab successfully demonstrated how to implement push notifications in a Flutter application using **Firebase Cloud Messaging (FCM)**. The integration process covered essential steps such as configuring Firebase, requesting notification permissions, retrieving the device token, and handling incoming messages across different application states.

Through practical implementation, it was observed that push notifications can be received both when the application is active (foreground) and when it is running in the background. The ability to display notifications in the system tray, as well as within the application interface using dynamic UI updates and popup dialogs, highlights the flexibility and power of FCM.

Additionally, the use of a unique device token enabled targeted message delivery from the Firebase Console, reinforcing the concept of personalized communication between the server and client devices.

Overall, this lab provided a solid foundation in real-time messaging for mobile applications. It emphasized the importance of push notifications in enhancing user engagement, delivering timely information, and improving the overall functionality of modern mobile systems.